

Chapter 2: VECTOR (Most Important STL Container)

If you master **Vector**, you'll use it in 60–70% of DSA problems.

What is Vector?

A **dynamic array**.

Normal array:

```
int arr[5];
```

Size is fixed.

Vector:

```
vector<int> v;
```

Size can grow and shrink automatically.

Why Use Vector?

Problems with arrays:

- Fixed size
- Cannot easily insert/delete
- Need size known beforehand

Vector solves these.

Declaration

Empty Vector

```
vector<int> v;
```

Vector with Size

```
vector<int> v(5);
```

Output:

```
0 0 0 0 0
```

Size + Value

```
vector<int> v(5,10);
```

Output:

```
10 10 10 10 10
```

Initialize Directly

```
vector<int> v = {1,2,3,4,5};
```

Copy Vector

```
vector<int> v1 = {1,2,3};
```

```
vector<int> v2(v1);
```

Access Elements

Using Index

```
cout << v[0];
```

Using at()

```
cout << v.at(0);
```

Safer than `[]`.

Important Functions

1. push_back()

Insert at end.

```
vector<int> v;  
  
v.push_back(10);  
v.push_back(20);  
v.push_back(30);
```

Result:

```
10 20 30
```

Complexity

```
O(1)
```

(amortized)

2. pop_back()

Remove last element.

```
v.pop_back();
```

Before:

```
10 20 30
```

After:

```
10 20
```

Complexity

```
O(1)
```

3. size()

Returns number of elements.

```
cout << v.size();
```

Output:

```
3
```

Complexity

```
O(1)
```

4. empty()

Checks if vector is empty.

```
if(v.empty())
```

Returns:

```
true  
false
```

5. clear()

Deletes all elements.

```
v.clear();
```

After:

```
size = 0
```

6. front()

Returns first element.

```
cout << v.front();
```

Example:

```
1 2 3 4
```

Output:

```
1
```

7. back()

Returns last element.

```
cout << v.back();
```

Output:

```
4
```

8. resize()

Changes size.

```
vector<int> v={1,2,3};  
  
v.resize(5);
```

Result:

```
1 2 3 0 0
```

Example:

```
v.resize(2);
```

Result:

```
1 2
```

9. assign()

Replace entire vector.

```
vector<int> v;  
v.assign(5,100);
```

Result:

```
100 100 100 100 100
```

10. insert()

Insert element at position.

```
vector<int> v={1,2,4};  
v.insert(v.begin()+2,3);
```

Result:

```
1 2 3 4
```

Complexity

```
O(n)
```

11. erase()

Remove element.

```
v.erase(v.begin()+2);
```

Before:

1 2 3 4

After:

1 2 4

Complexity

$O(n)$

Erase Range

```
v.erase(v.begin()+1,v.begin()+4);
```

Before:

1 2 3 4 5

After:

1 5

12. swap()

Swap vectors.

```
v1.swap(v2);
```

13. capacity()

Returns allocated memory.

```
cout << v.capacity();
```

Interview concept.

Size $\neq$ Capacity

Example:

```
size = 5  
capacity = 8
```

14. shrink_to_fit()

Reduce capacity.

```
v.shrink_to_fit();
```

Rarely used.

15. data()

Returns pointer to first element.

```
int* ptr = v.data();
```

Advanced.

Iterating Through Vector

Method 1

```
for(int i=0;i<v.size();i++)  
{
```

```
    cout<<v[i]<<" ";  
}
```

Method 2

Range Based Loop

```
for(auto x:v)  
{  
    cout<<x<<" ";  
}
```

Method 3

Iterator

```
for(auto it=v.begin();it!=v.end();it++)  
{  
    cout<<*it<<" ";  
}
```

Iterators

Very Important

begin()

Points first element.

```
v.begin()
```

end()

Points after last element.

```
v.end()
```

rbegin()

Reverse begin.

```
v.rbegin()
```

rend()

Reverse end.

```
v.rend()
```

Reverse Printing:

```
for(auto it=v.rbegin();it!=v.rend();it++)
{
    cout<<*it<<" ";
}
```

2D Vector

Declaration

```
vector<vector<int>> mat;
```

Initialize

```
vector<vector<int>> mat(3,vector<int>(4,0));
```

Creates:

```
0 0 0 0
0 0 0 0
0 0 0 0
```

Access

```
mat[1][2]
```

Time Complexity Table

Operation	Complexity
push_back	O(1)
pop_back	O(1)
access	O(1)
front	O(1)
back	O(1)
size	O(1)
insert middle	O(n)
erase middle	O(n)
clear	O(n)

Interview Questions

Q1

Input N numbers.

Store in vector.

Print them.

Q2

Find sum of all vector elements.

Input:

```
5
```

```
1 2 3 4 5
```

Output:

15

Q3

Find largest element.

Input:

1 7 3 9 2

Output:

9

Q4

Reverse a vector without using `reverse()`.

Q5

Delete all even numbers.

Input:

1 2 3 4 5 6

Output:

1 3 5

Q6

Insert 100 at index 2.

Input:

```
1 2 3 4 5
```

Output:

```
1 2 100 3 4 5
```

Q7

Rotate vector left by 1.

Input:

```
1 2 3 4 5
```

Output:

```
2 3 4 5 1
```

Q8 (Interview)

Frequency Count

Input:

```
1 2 2 3 1 1
```

Output:

```
1 -> 3  
2 -> 2  
3 -> 1
```

(Hint: later we'll use `unordered_map`.)

Mini STL Challenge

Input:

8

5 1 7 3 2 9 4 6

Tasks:

1. Store in vector.
2. Print vector.
3. Print size.
4. Print first and last element.
5. Remove last element.
6. Insert 100 at index 3.
7. Delete element at index 2.
8. Reverse print using iterator.

Revision Sheet

Create

```
vector<int> v;
```

Add

```
push_back()
```

Remove

```
pop_back()  
erase()  
clear()
```

Access

```
[]  
at()  
front()  
back()
```

Size

```
size()  
capacity()  
empty()
```

Modify

```
insert()  
resize()  
assign()  
swap()
```

Iterators

```
begin()  
end()  
rbegin()  
rend()
```

Complexity

```
Access = O(1)  
push_back = O(1)  
insert = O(n)  
erase = O(n)
```

Assignment Before Next Chapter

Solve:

1. Reverse vector.
2. Find max element.
3. Find min element.
4. Remove duplicates from sorted vector.

5. Left rotate by K positions.
6. Right rotate by K positions.
7. Merge two vectors.
8. Frequency count.